



Advanced Database Systems:

Part III.3a Key-Value Stores

Reinhard Pichler

Institute of Logic and Computation
"Databases & Artificial Intelligence" Group

Summer Semester 2026

References

❖ Articles:

- Justin Sheehy and David Smith. Bitcask: A Log-Structured Hash Table for Fast Key/Value Data. Basho Technologies, 2010. <https://riak.com/assets/bitcask-intro.pdf> (inspired by Eric Brewer)
- Patrick E. O'Neil, Edward Cheng, Dieter Gawlick, Elizabeth J. O'Neil: The Log-Structured Merge-Tree (LSM-Tree). Acta Informatica 33(4): 351-385 (1996)

❖ Book chapter:

- Martin Kleppmann: Designing Data-Intensive Applications: The big ideas behind reliable, scalable, and maintainable systems, Chapter 3: "Storage and Retrieval" O'Reilly, 2017

❖ Unless stated otherwise, copyright of all figures is with one of these resources.

Contents

❖ **Basic Principles of K/V Stores**

❖ Log-Structured Hash-Tables (Bitcask)

- Main ideas
- Basic operations
- Strengths and weaknesses

❖ Log-Structured Merge-Trees (LSM-Trees)

- Principal layout
- Basic operations
- Some optimisations
- Strengths and weaknesses

Key-Value Stores

❖ Main Characteristics:

- store values as arrays of byte, indexed by keys
- Support a limited collection of data types, e.g.: text, JSON, XML, list, binary, etc.
- semantics of the value (= structured or unstructured data) is only known to the application
- no complex queries (mostly look-up by key)
- focus on scalability, high availability and performance
- typically write-heavy workloads
- Pioneer: Amazon's DynamoDB (for shopping carts)

Typical Use Cases

- a "key" is naturally available and used for data access;
- requirements match K/V store characteristics (scalability, high availability, performance).

Application Type	Key	Value
Session	User / Session ID	Session Data
Logs	Date, Date/Time	Log Entry
Sensor	Date, Date/Time	Sensor Updates
User Data	Login, email	User Attributes
Advertising	Campaign ID	Ad content

Common Storage Types for Database Systems

❖ B⁺-trees:

- flexible index structure, widely used in DBMSs
- logarithmic time for insert, update, delete, search

❖ Log-structured (append-only) files:

- fast writes: constant time
- slow reads: linear time

❖ Goal for K/V stores:

- fast writes: constant time
- without sacrificing on reads

❖ Solution: Log-structured file + index structure

Contents

- ❖ Basic Principles of K/V Stores
- ❖ **Log-Structured Hash-Tables (Bitcask)**
 - Main ideas
 - Basic operations
 - Strengths and weaknesses
- ❖ Log-Structured Merge-Trees (LSM-Trees)
 - Principal layout
 - Basic operations
 - Some optimisations
 - Strengths and weaknesses

Bitcask Origin

- ❖ Originally developed for the **Riak K/V Store**:
<https://riak.com/products/riak-kv/>
- ❖ Built to be generic and can be used by other applications
- ❖ **Embedded database**: application links to the Bitcask library directly, no separate daemon/service to connect to over the network
- ❖ Riak K/V Store is a **distributed database server**:
Bitcask is used (by default) as local storage on each node.

Goals

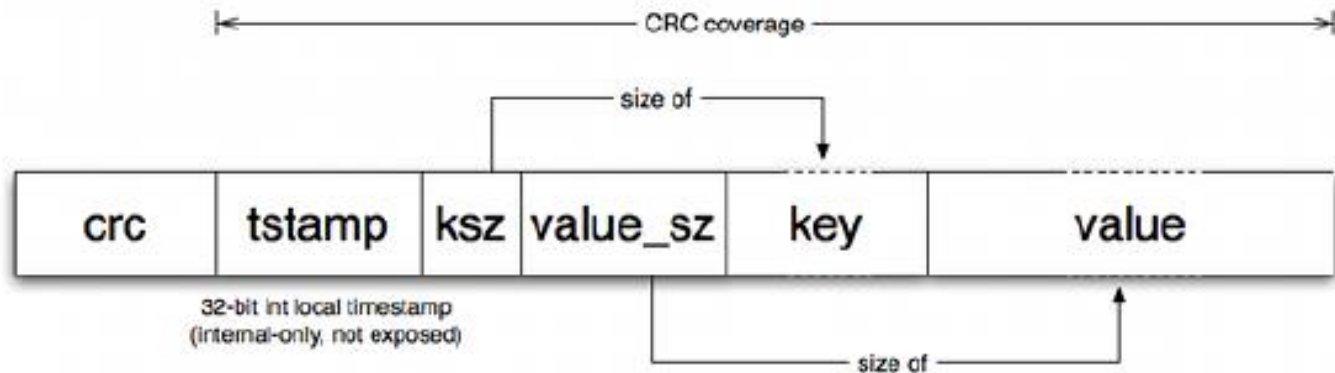
- ❖ low read and write **latency**
- ❖ high **write throughput** (especially when writing an incoming stream of random items)
- ❖ ability to handle large datasets without degradation
- ❖ "**crash friendliness**": no replay of write operations needed since the data files are the commit log; recomputing the hash table may be time-consuming and requires some measures.
- ❖ ease of **backup and restore**
- ❖ a relatively **simple**, understandable code structure
- ❖ predictable behaviour also under heavy load

Design Principles

- ❖ A **Bitcask instance is a directory**. At any moment, only one file is **"active"** in this directory and can be written to.
- ❖ When the active file reaches a **size threshold**, it is closed and a new active file will be created.
- ❖ Once a file is closed, it is considered **immutable** and will not be opened for writing again.
- ❖ With each write, a **new entry is appended** to the active file. For delete, a special **tombstone** value is written.
- ❖ A **hash map in memory** allows fast access via the key to the most recent value in one of the data files.

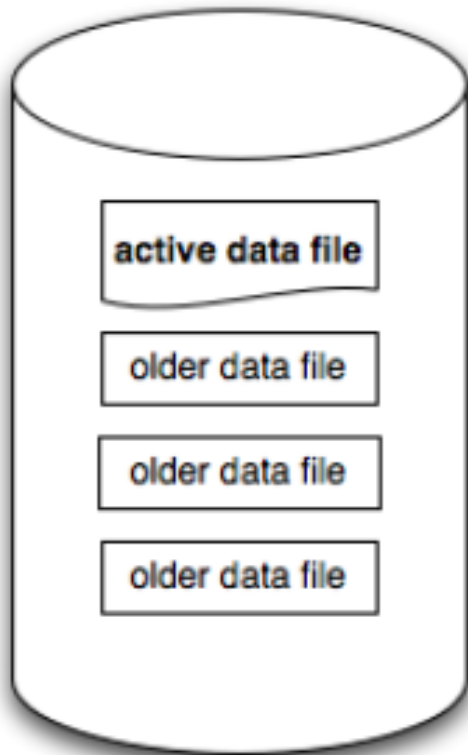
Format of a Single Entry

- ❖ CRC (= Cyclic Redundancy Check):
error detecting code to detect corrupted entries
- ❖ tstamp: time stamp
- ❖ ksz: key size
- ❖ value_sz: value size
- ❖ key
- ❖ value



A Bitcask on Disk

Bitcask directory:

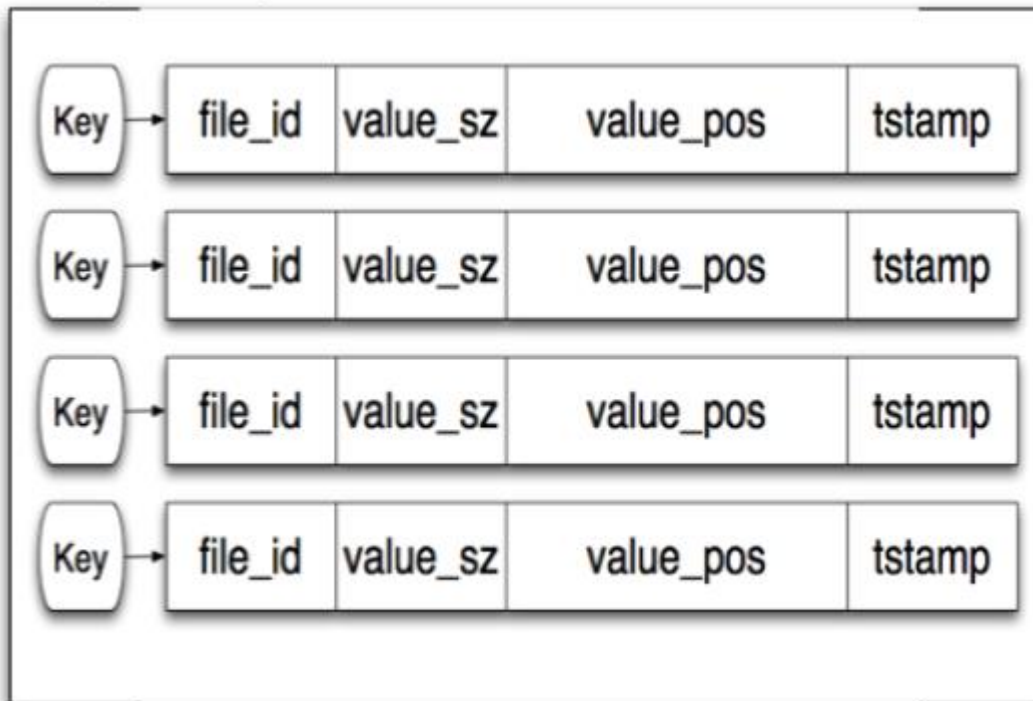


data file:

crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value
crc	tstamp	ksz	value_sz	key	value

Hash Table in Memory

- ❖ in Bitcask: called **keydir**
- ❖ a **hash table** that maps every key to a fixed-sized structure:
file_id, offset, value size, timestamp of most recent value



Basic Operations

- ❖ insert (= put)
- ❖ update (= put)
- ❖ delete
- ❖ search (= get)
- ❖ compaction (= "merging") of the data files
- ❖ further operations:
 - open/close Bitcask
 - list_keys
 - fold: apply user-supplied function to all live entries
 - sync: flush buffered writes to stable disk storage

Put Operation (Insert, Update)

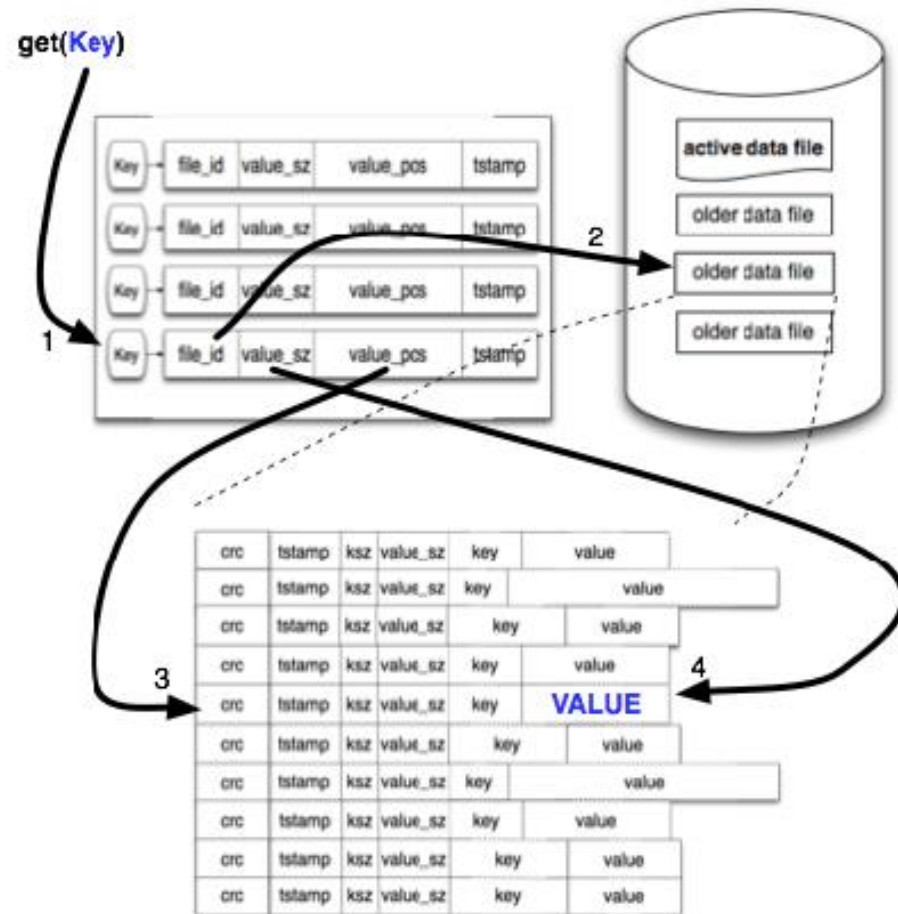
- ❖ The **put** operation is called with a key/value pair.
- ❖ For this key/value pair, **an entry for the active data file** is generated:
 - Add timestamp, key_size, and value_size (4-byte integers).
 - Compute and add the CRC value of these 3 integers plus key and value.
- ❖ **Append the entry** to the active data file. If this would exceed the size threshold, close it and create a new active data file.
- ❖ Check if the key already exists **in the keydir**:
 - If not (= insert): insert the key + metadata into the keydir
 - If so (= update): update the metadata of this key.

Delete Operation

- ❖ Search for the given key in the keydir
- ❖ If it exists:
 - Create a **new entry for the active data file** similarly to a put operation; but now with the special **tombstone** value.
 - **Append the entry** to the active data file (again: possibly a new active data file has to be created if the size threshold would be exceeded)
 - **Remove the key from the keydir** (i.e., no pointer to the entry with the tombstone is needed)

Get Operation

- ❖ Only **point-queries**, i.e.: find the value for a given key
- ❖ File, offset, and size of the most recent value for the key are **obtained from the keydir**.
- ❖ Then a **single disk access** suffices to read out the value.



Compaction (referred to as "Merging")

❖ Problem with log-structured (append-only) storage:

- Updates and deletes are handled by **appending new entries**.
- The storage may contain invalid entries, because a more recent value (possibly a tombstone) exists for the key.
- Without counter-measures, **storage may grow indefinitely**.

❖ Solution:

- **Compaction (= "merging")**,
i.e., scan the contents of several data files and delete all but the **most recent value for every key**. And delete all entries if a more recent **tombstone** exists for the same key.

Merging

- ❖ The merge process iterates over **all non-active data files** and produces new data files with only the latest value version of each present key.
- ❖ The **metadata of all keys in the keydir** has to be updated to point to the new location of the latest value.
- ❖ During merging, a **hint file** is created next to each data file:
 - The hint file contains **no values**;
 - it contains the position and size of every value.
 - This is useful to **speed up start-up and recovery**.

Strengths of Bitcask (goals revisited)

- ❖ low read and write latency:
 - read: hash table in memory points to locations on disk; hence, at most one disk access per read operation;
 - write: simply append data
- ❖ high write throughput (especially when writing an incoming stream of random items): only sequential writes to disk
- ❖ ability to handle large datasets without degradation: cost of get/put/delete is largely independent of size of the data
- ❖ ease of backup and restore:
 - for backup: just copy the data files
 - for restore: copy the data files back and reconstruct the keydir using the hint files.

Strengths of Bitcask (goals revisited)

❖ "crash friendliness":

- no replay of write operations needed since the data files are the commit log;
- only the last entries in the active data file might be partially written => analyse the CRC to ensure consistency
- the recomputing the keydir may be time-consuming but the hint files help to speed it up.

❖ a relatively simple, understandable code structure:

Bitcask is conceptually easy, with a simple interface, and with a clear and easy to understand file structure

❖ predictable behaviour also under heavy load:

essentially constant time required for a single put/get/delete

Weaknesses of Bitcask

- ❖ High memory usage:

- all keys plus meta-information is stored in RAM.
- Hence, Bitcask is best suited for scenarios, where the number of keys is not too huge and with frequent writes.

- ❖ Only point queries are supported. What about range queries?

Contents

- ❖ Basic Principles of K/V Stores
- ❖ Log-Structured Hash-Tables (Bitcask)
 - Main ideas
 - Basic operations
 - Strengths and weaknesses
- ❖ **Log-Structured Merge-Trees (LSM-Trees)**
 - Principal layout
 - Basic operations
 - Some optimisations
 - Strengths and weaknesses

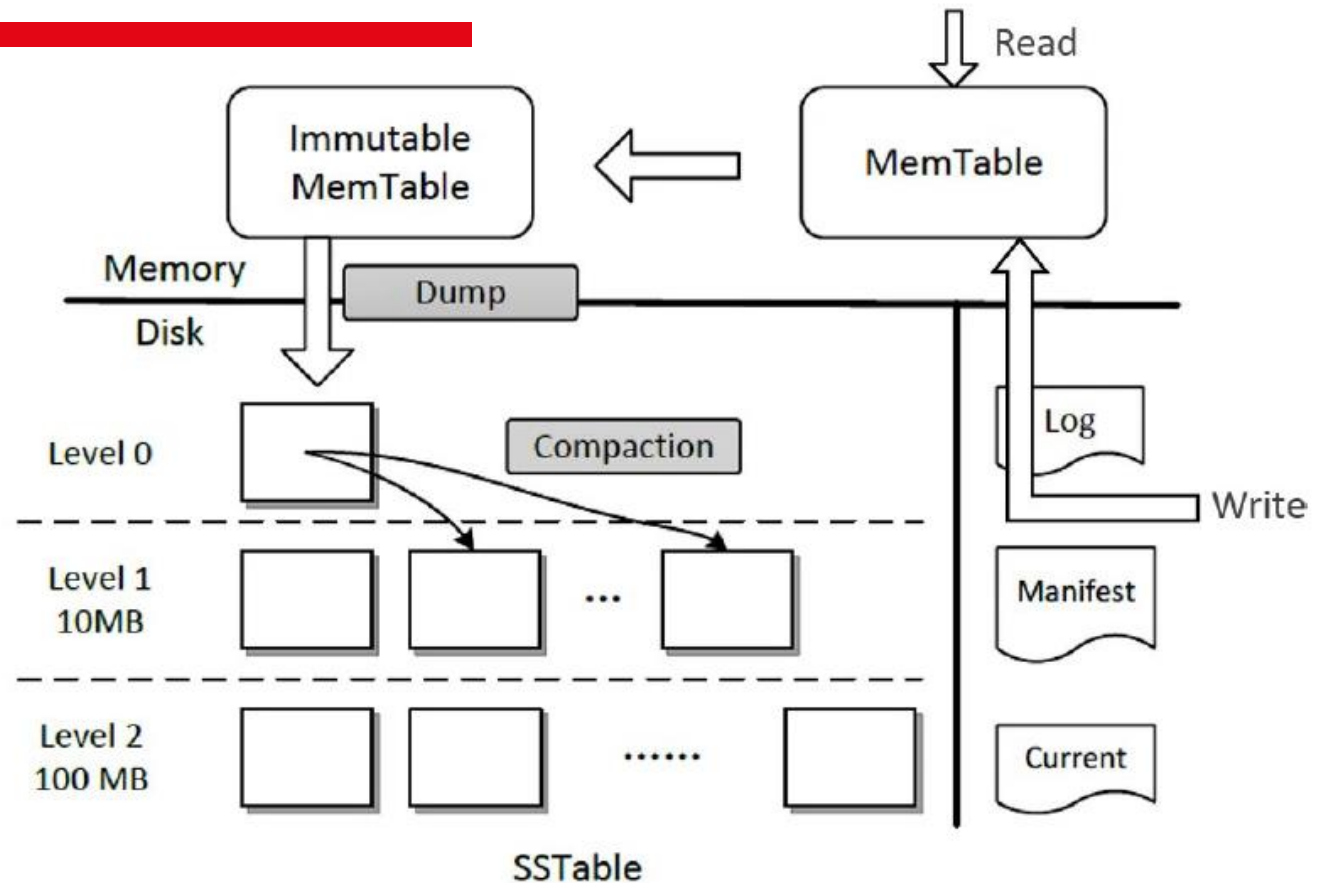
Basic Principles

- ❖ Key-Value pairs are **sorted by the key**.
- ❖ Writes are out-of-place as with log-structured hash-tables.
- ❖ The most recent writes are kept in RAM (= "**memtable**") in some sorted data structure (e.g., skip list, red-black tree, ...)
- ❖ Data on disk is stored in sorted files (= sorted string tables, **SSTables** for short)
- ❖ The SSTables are arranged in "**levels**" (level 0 – level k-1), with **more recent data in higher levels**
- ❖ The **size of the levels grows exponentially**, e.g.:
level 1, 2, 3, ...: 10MB, 100MB, 1000MB, ...

Use of LSM-Trees

- ❖ LSM-Trees can be used as **storage engine of various types of database systems**, e.g.: for relational data, use the primary key as key and the remaining attributes as value
- ❖ LSM-Trees are used in many **state-of-the art K/V-stores**:
 - Google: LevelDB (SSTables originally introduced in BigTable)
 - Facebook: RocksDB
 - Amazon: DynamoDB
 - LinkedIn: Voldemort
 - Apache: Cassandra (also in HBase)
- ❖ Some are **open source** (e.g., LevelDB, RocksDB)
- ❖ **Riak K/V-Store**: allows to switch from Bitcask to LevelDB

LevelDB Architecture



Source: <https://gifted-dl.medium.com/deep-dive-on-read-write-and-space-amplification-in-ssds-and-lsm-storage-engines-and-what-makes-4a1e15fc6f0e>

Components of LevelDB

- ❖ The **mutable memtable** is the entry point for reads and writes.
- ❖ If the mutable memtable reaches its threshold size, it is made immutable and a new mutable memtable is created.
- ❖ The **immutable memtable** is flushed to disk (Level-0).
- ❖ The memtables are in RAM and would be lost in a crash. All writes are therefore written into the **Log** file (write-ahead log).
- ❖ The **Manifest** file contains (in the form of append-only) the history of all SSTables (e.g., remove fileX, add fileY to Level-Z) and is therefore crucial in case of recovery after a crash.
- ❖ When Manifest grows too big, a new Manifest file is created with a snapshot of the current state of the database.
- ❖ The **Current** file just contains the name of the current Manifest.

Basic Operation

❖ Internal operations:

- Flush
- Compaction (= merging)

❖ External operations:

- Put (Insert, Update)
- Delete
- Get (= point queries)
- Scan (= range queries)

Flush

- ❖ The immutable memtable serves as a **write buffer** so as not to block writes during flush.
- ❖ The data from the immutable memtable is converted into SSTable format and written to disk. It is then part of **level-0**.
- ❖ Level-0 may contain several SSTables and Level-0 is the only level that may contain SSTables with **overlapping key ranges**.
- ❖ Motivation: LSM-trees are write-optimized. To **avoid write stalls**, flush of the immutable memtable has to complete before the mutable memtable reaches the threshold size again.
- ❖ Only when SSTables from level-0 are compacted with (some or all of) the SSTables from level-1, **disjointness of the key ranges** of different SSTables on the same level is guaranteed.

Compaction

- ❖ As with log-structured hash tables: due to the out-of-place realization of writes (update, delete), the SSTables may contain **invalid or deleted values**. To be reduced via compaction.
- ❖ **Trade-off of compaction**: compaction reduces read and space amplification at the cost of write amplification.
- ❖ When a level reaches its size threshold, all or part of the data of this level is **moved to the next level**, where it is merged with those SSTables whose key range overlaps.

Various Compaction Strategies

- ❖ When is a compaction step **triggered**?
 - In particular, when the size threshold of a level is reached
 - other, e.g.: timer/counter for tombstones, space amplification
- ❖ Data layout on **individual levels**:
 - **Leveling**: non-overlapping SSTables, favours reads, less space
 - **Tiering**: overlapping SSTables allowed, favours writes
- ❖ Compaction **granularity**:
 - merge one / some / all SSTables of a level to the next level
 - merge SSTables of a level into 1 file and move to next level
- ❖ How to **select** one / some SSTables for compaction?
 - oldest / coldest (least recently accessed) file in a level
 - round-robin

Put Operation (Insert, Update)

- ❖ The put operation is called with a key-value pair.
- ❖ The key is **searched for in the sorted data structure** in the mutable memtable.
 - **If the key exists** there (= update), the current value is overwritten with the value of the entered key-value pair.
 - **If the key does not exist**, then the key-value pair is entered into the sorted data structure at the right place.
 - Note that even if the entered key does not exist in the mutable memtable, **this put operation can be either an insert or an update** (with an older value for the same key existing in the immutable memtable or in some SSTable on disk).

Delete Operation

- ❖ The delete is handled **analogously to the put operation**; but now we associate a **tombstone as "value"** with the given key.
- ❖ That is, the given key is searched for in the mutable memtable:
 - If it exists, the current value is replaced by the tombstone.
 - If it does not exist, a new key-value pair (with the tombstone as value) is inserted into the sorted data structure in the mutable memtable.
 - Note that, even if the given key does not exist in the mutable memtable, a tombstone entry is required, because the key could exist in the immutable memtable or in some SSTable.

Get Operation

- ❖ Search for the given key in the **mutable memtable**.
- ❖ If not found, search in the **immutable memtable**.
- ❖ If not found, search in the **SSTables**:
 - Search **levels in ascending order**, starting at level-0.
 - **level = 0**: most recent SSTables first (**overlapping ranges!**)
 - **levels > 0**: only one SSTable may contain the key (no overlapping ranges), no duplicate key inside an SSTable.
- ❖ Either return value of found key or "not found".
- ❖ **Problem**: too many SSTables and too many blocks inside the SSTables have to be read from disk.

Optimisations

- ❖ Bloom filter for every SSTable:
 - no false negatives and low probability of false positives
 - In particular: allows to skip most SSTables
- ❖ Index block (containing "fence pointers"):
 - sparse index for the blocks inside an SSTables, e.g.: min key value of each block plus pointer to the block
- ❖ Typical flow when checking an SSTable:
 - Bloom filter: could the searched for key be contained?
 - If yes, obtain start of relevant block by a binary search in the fence pointer index.
 - Load the block and search for the key with binary search.

Scan (= Range Query)

- ❖ For a **given interval** $[k_{\min}, k_{\max}]$ of key values, all existing key-value pairs (k_i, v_i) in the database (with the newest value for each key) **with** $k_{\min} \leq k_i \leq k_{\max}$ have to be returned.
- ❖ This requires a search in the memtables as well as in every SSTable whose key range overlaps with $[k_{\min}, k_{\max}]$.
- ❖ Typically, this is realized by an **iterator on each level** and a **merge-like operation** that determines for every key found the newest value (i.e., an actual data value or a tombstone).

Strengths of LSM-Trees

❖ Compared with B⁺-trees:

- higher write throughput (mostly sequential writes)
- much better with write bursts (memtable + Log, flush)
- SSTables are "compression friendly": immutable, sorted, repeated prefixes in the same SSTable

❖ Compared with log-structured hash-tables:

- not limited by RAM
- efficient evaluation of range queries (and analytical queries in general)

Weaknesses of LSM-Trees

❖ Compared with B+-trees:

- less efficient reads: look-up may require reading of memtables, several SSTables (in case of tiering: even on the same level), Bloom filters
- more complex evaluation of range queries: data fragmented across levels, merge required due to out-of-place writes
- compaction overhead: CPU, I/O-bandwidth, cache

❖ Compared with log-structured hash-tables:

- more complex design (e.g., hierarchical compaction)
- less efficient evaluation of point queries

Summary

- ❖ main characteristics, typical use cases of K/V-stores
- ❖ Log-structured hash-table (Bitcask):
 - basic principles
 - put, delete, get; compaction
 - strengths and weaknesses
- ❖ Log-structured merge-trees (LSM-trees; in particular, LevelDB):
 - basic principles
 - put, delete, get, scan; flush, compaction
 - optimisations
 - strengths and weaknesses